# Лабораторная работа №4. PKI, HTTPS и TLS 1.3

**Вариант 4**

## Цель работы

На собственном тестовом CA выпустить цепочку сертификатов X.509 с расширениями, настроить HTTPS-сервер, принимающий только TLS 1.3, проверить цепочку доверия и отзыв, захватить рукопожатие TLS 1.3 и разобрать его в Wireshark, в том числе после расшифровки через журнал ключей (SSLKEYLOGFILE).

## Задание варианта 4

| Параметр | Значение |
|---|---|
| Ключ корня / промежуточного CA | Ed25519 |
| Ключ сервера | ECDSA P-384 |
| Группа ECDHE | P-384 (secp384r1, код 24) |
| Набор шифров TLS 1.3 | TLS_AES_256_GCM_SHA384 (0x1302) |

Задача варианта (mTLS): nginx с ssl_verify_client on и ssl_client_certificate (файл с корнем и промежуточным CA). В захвате показать CertificateRequest (type 13) и клиентский Certificate. Без клиентского сертификата соединение отвергается.

## Стенд

Работа выполнялась на Windows 11 через WSL2. В WSL установилась более новая версия Ubuntu, поэтому версии программ отличаются от стенда методички.

| Компонент | В методичке | Фактически |
|---|---|---|
| ОС | Ubuntu 24.04 | Ubuntu 26.04.1 LTS (WSL2) |
| OpenSSL | 3.0.13 | 3.5.5 |
| curl | 8.5.0 | 8.18.0 |
| nginx | 1.24.0 (nginx-light) | 1.28.3 (nginx; nginx-light в репозитории нет) |
| tshark | 4.2.2 | 4.6.4 |
| tcpdump | — | 4.99.6 |

Изменения относительно методички:

– в ca.cnf указано default_md = default вместо sha256, так как ключ промежуточного CA — Ed25519, у которого хэш встроен в алгоритм подписи;
– при создании сертификатов CA с ключом Ed25519 флаг -sha256 не используется;
– в запросах curl добавлен флаг --curves P-384: OpenSSL 3.5 по умолчанию первым предлагает группу X25519MLKEM768, а сервер принимает только P-384, и без флага рукопожатие шло бы через HelloRetryRequest;
– пути в конфигурации nginx — /home/tim/lab4/...

## 1. Корневой CA

Сгенерирован ключ Ed25519 и самоподписанный корневой сертификат с расширениями basicConstraints (CA:TRUE, pathlen:1) и keyUsage (keyCertSign, cRLSign).

```
mkdir -p ~/lab4/root ~/lab4/inter ~/lab4/server ~/lab4/client && cd ~/lab4
openssl genpkey -algorithm ED25519 -out root/root.key && chmod 600 root/root.key
openssl req -new -x509 -key root/root.key -days 3650 \
  -subj '/C=BY/O=Lab PKI/CN=Lab Root CA' \
  -addext 'basicConstraints=critical,CA:TRUE,pathlen:1' \
  -addext 'keyUsage=critical,keyCertSign,cRLSign' \
  -addext 'subjectKeyIdentifier=hash' -out root/root.crt
ls -l root/
openssl x509 -in root/root.crt -noout -text | grep -E 'Signature Algorithm|
Issuer|Subject:|Not After|CA:|Sign'
```

```
tim@Tim:~/lab4$ ls -l root/ && openssl x509 -in ro
total 8
-rw-r--r-- 1 tim tim 607 Nov  9 20:05 root.crt
-rw------- 1 tim tim 119 Nov  9 20:05 root.key
        Signature Algorithm: ED25519
        Issuer: C=BY, O=Lab PKI, CN=Lab Root CA
            Not After : Nov  6 17:05:45 2036 GMT
        Subject: C=BY, O=Lab PKI, CN=Lab Root CA
                CA:TRUE, pathlen:1
                Certificate Sign, CRL Sign
    Signature Algorithm: ED25519
    Signature Value:
```

Закрытый ключ корня имеет права 600. У корневого сертификата поля Issuer и Subject совпадают, так как он самоподписанный. Алгоритм подписи — ED25519, разрешена роль CA с pathlen:1, ключ может подписывать сертификаты и списки отзыва.

## 2. Промежуточный CA

Сгенерирован ключ Ed25519 промежуточного CA, создан запрос на сертификат и подписан корневым CA. Ограничение pathlen:0 запрещает промежуточному CA выпускать другие CA.

```
cat > inter/inter.ext <<'EOF'
basicConstraints=critical,CA:TRUE,pathlen:0
keyUsage=critical,keyCertSign,cRLSign
```

```
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid:always
EOF
openssl genpkey -algorithm ED25519 -out inter/inter.key && chmod 600
inter/inter.key
openssl req -new -key inter/inter.key -subj '/C=BY/O=Lab PKI/CN=Lab Intermediate
CA' -out inter/inter.csr
openssl x509 -req -in inter/inter.csr -CA root/root.crt -CAkey root/root.key \
  -CAcreateserial -days 1825 -extfile inter/inter.ext -out inter/inter.crt
openssl x509 -in inter/inter.crt -noout -text | grep -E 'Signature Algorithm|
Issuer|Subject:|CA:|Sign'
```

```
tim@Tim:~/lab4$ openssl x509 -req -in inter/inter.csr -CA root/root.crt -CAkey root/roo
s 1825 -extfile inter/inter.ext -out inter/inter.crt
Certificate request self-signature ok
subject=C=BY, O=Lab PKI, CN=Lab Intermediate CA
tim@Tim:~/lab4$ openssl x509 -in inter/inter.crt -noout -text | grep -E 'Signature Algo
Sign'
        Signature Algorithm: ED25519
        Issuer: C=BY, O=Lab PKI, CN=Lab Root CA
        Subject: C=BY, O=Lab PKI, CN=Lab Intermediate CA
                CA:TRUE, pathlen:0
                Certificate Sign, CRL Sign
    Signature Algorithm: ED25519
    Signature Value:
```

Издатель промежуточного сертификата — Lab Root CA, в Basic Constraints указано CA:TRUE, pathlen:0.

## 3. База промежуточного CA

Созданы файлы базы (index.txt, serial, crlnumber) и конфигурация ca.cnf. Выпуск через openssl ca записывает сертификаты в базу, что нужно для последующего отзыва.

```
cd ~/lab4/inter && touch index.txt && echo 01 > crlnumber && echo 1000 > serial
&& cd ~/lab4
cat > inter/ca.cnf <<'EOF'
[ ca ]
default_ca = CA_default
[ CA_default ]
dir               = /home/tim/lab4/inter
database          = $dir/index.txt
new_certs_dir     = $dir
serial            = $dir/serial
crlnumber         = $dir/crlnumber
certificate       = $dir/inter.crt
private_key       = $dir/inter.key
default_md        = default
default_days      = 365
default_crl_days  = 30
unique_subject    = no
copy_extensions   = none
policy            = policy_any
[ policy_any ]
commonName        = supplied
EOF
ls inter/
```

```
tim@Tim:~/lab4$ ls inter/ && cat inter/serial inter/crlnumber
ca.cnf  crlnumber  index.txt  inter.crt  inter.csr  inter.ext  inter.key  serial
1000
01
```

## 4. Серверный и клиентский сертификаты

Сгенерирован ключ сервера ECDSA P-384, сертификат выпущен промежуточным CA с SAN (DNS:localhost, IP:127.0.0.1) и EKU serverAuth. Для задачи варианта (mTLS) аналогично выпущен клиентский сертификат с EKU clientAuth.

```
cat > server/server.ext <<'EOF'
basicConstraints=critical,CA:FALSE
keyUsage=critical,digitalSignature
extendedKeyUsage=serverAuth
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid:always
subjectAltName=DNS:localhost,IP:127.0.0.1
EOF
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out
server/server.key && chmod 600 server/server.key
openssl req -new -key server/server.key -subj '/C=BY/O=Lab PKI/CN=localhost' -out
server/server.csr
openssl ca -config inter/ca.cnf -batch -in server/server.csr -extfile
server/server.ext -notext -out server/server.crt
```

```
tim@Tim:~/lab4$ openssl ca -config inter/ca.cnf -batch -in server/server.csr -extfi
out server/server.crt
Using configuration from inter/ca.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName           :PRINTABLE:'BY'
organizationName      :ASN.1 12:'Lab PKI'
commonName            :ASN.1 12:'localhost'
Certificate is to be certified until Nov  9 17:14:59 2027 GMT (365 days)

Write out database with 1 new entries
Database updated
```

```
cat > client/client.ext <<'EOF'
basicConstraints=critical,CA:FALSE
keyUsage=critical,digitalSignature
extendedKeyUsage=clientAuth
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid:always
EOF
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out
client/client.key && chmod 600 client/client.key
openssl req -new -key client/client.key -subj '/C=BY/O=Lab PKI/CN=lab-client' -
out client/client.csr
openssl ca -config inter/ca.cnf -batch -in client/client.csr -extfile
client/client.ext -notext -out client/client.crt
```

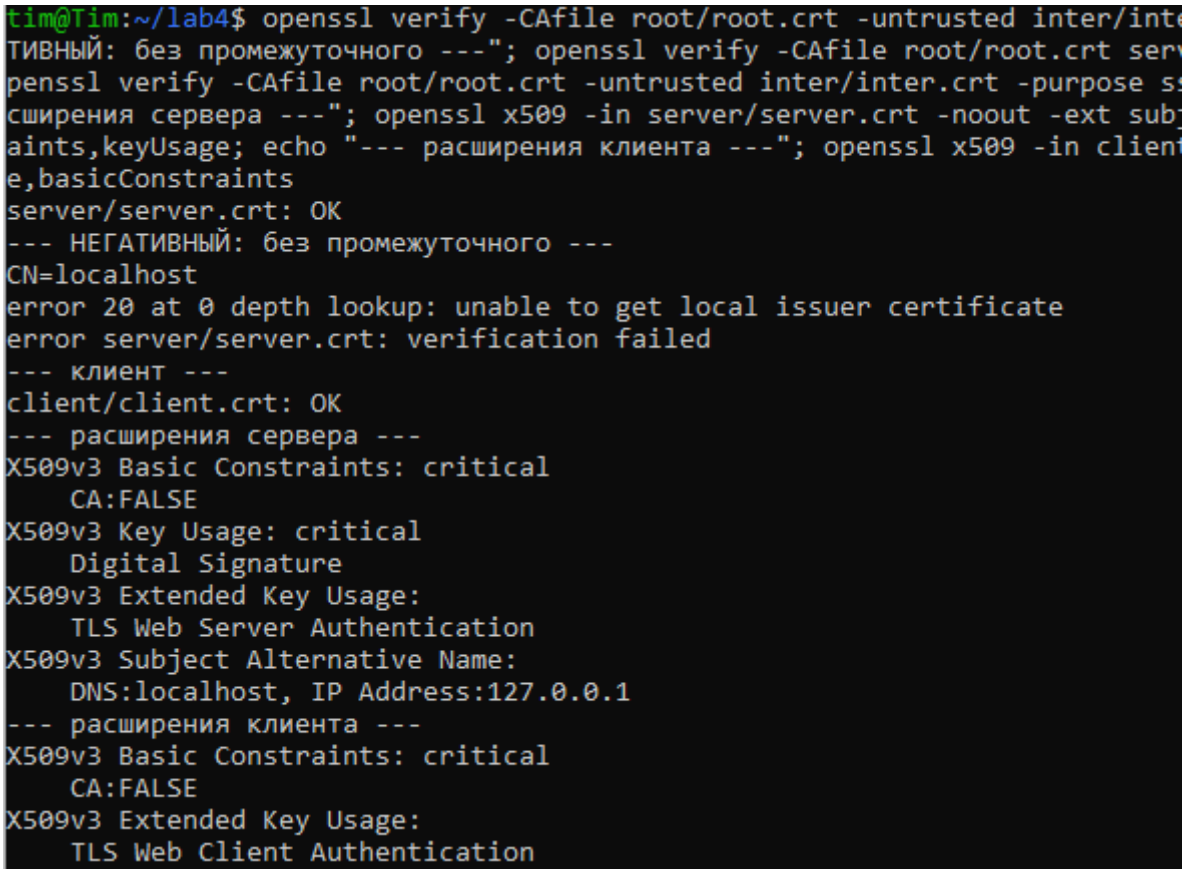
```
tim@Tim:~/lab4$ openssl ca -config inter/ca.cnf -batch -in client/client.csr -extfile client/client.
out client/client.crt
Using configuration from inter/ca.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName           :PRINTABLE:'BY'
organizationName      :ASN.1 12:'Lab PKI'
commonName            :ASN.1 12:'lab-client'
Certificate is to be certified until Nov  9 17:16:22 2027 GMT (365 days)

Write out database with 1 new entries
Database updated
```

Оба сертификата записаны в базу промежуточного CA (сообщение Data Base Updated).

## 5. Проверка цепочки

```
openssl verify -CAfile root/root.crt -untrusted inter/inter.crt server/server.crt
openssl verify -CAfile root/root.crt server/server.crt
openssl verify -CAfile root/root.crt -untrusted inter/inter.crt -purpose
sslclient client/client.crt
openssl x509 -in server/server.crt -noout -ext
subjectAltName,extendedKeyUsage,basicConstraints,keyUsage
openssl x509 -in client/client.crt -noout -ext extendedKeyUsage,basicConstraints
```




С промежуточным сертификатом цепочка строится (OK). Без промежуточного сертификата (негативный тест) — unable to get local issuer certificate: путь от листа до корня не строится. Серверный сертификат содержит SAN localhost и 127.0.0.1, EKU TLS Web Server Authentication и CA:FALSE; клиентский — EKU TLS Web Client Authentication.

## 6. Настройка nginx (TLS 1.3 + mTLS)

Собран fullchain.pem (лист + промежуточный CA), который nginx отдаёт клиенту, и ca-chain.pem (промежуточный CA + корень) для проверки клиентских сертификатов.

```
cat server/server.crt inter/inter.crt > server/fullchain.pem
cat inter/inter.crt root/root.crt > client/ca-chain.pem
```

Файл /etc/nginx/conf.d/lab4.conf:

```
server {
    listen 8443 ssl;
    server_name localhost;
    ssl_certificate     /home/tim/lab4/server/fullchain.pem;
    ssl_certificate_key /home/tim/lab4/server/server.key;
```

```
    ssl_protocols TLSv1.3;
    ssl_conf_command Ciphersuites TLS_AES_256_GCM_SHA384;
    ssl_ecdh_curve P-384;
    ssl_session_tickets on;

    ssl_verify_client on;
    ssl_client_certificate /home/tim/lab4/client/ca-chain.pem;
    ssl_verify_depth 2;

    location / { return 200 'tls13 ok'; }
}

sudo nginx -t && sudo systemctl restart nginx
```

```
tim@Tim:~/lab4$ cat /etc/nginx/conf.d/lab4.conf && sudo nginx -t && sudo s
active nginx
server {
    listen 8443 ssl;
    server_name localhost;
    ssl_certificate     /home/tim/lab4/server/fullchain.pem;
    ssl_certificate_key /home/tim/lab4/server/server.key;
    ssl_protocols TLSv1.3;
    ssl_conf_command Ciphersuites TLS_AES_256_GCM_SHA384;
    ssl_ecdh_curve P-384;
    ssl_session_tickets on;

    ssl_verify_client on;
    ssl_client_certificate /home/tim/lab4/client/ca-chain.pem;
    ssl_verify_depth 2;

    location / { return 200 'tls13 ok'; }
}
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
active
```

## 7. Проверка клиентом и негативные тесты

Позитивный тест: запрос с доверенным корнем и клиентским сертификатом.

```
curl -v --curves P-384 --cacert root/root.crt --cert client/client.crt --key
client/client.key https://localhost:8443/
```

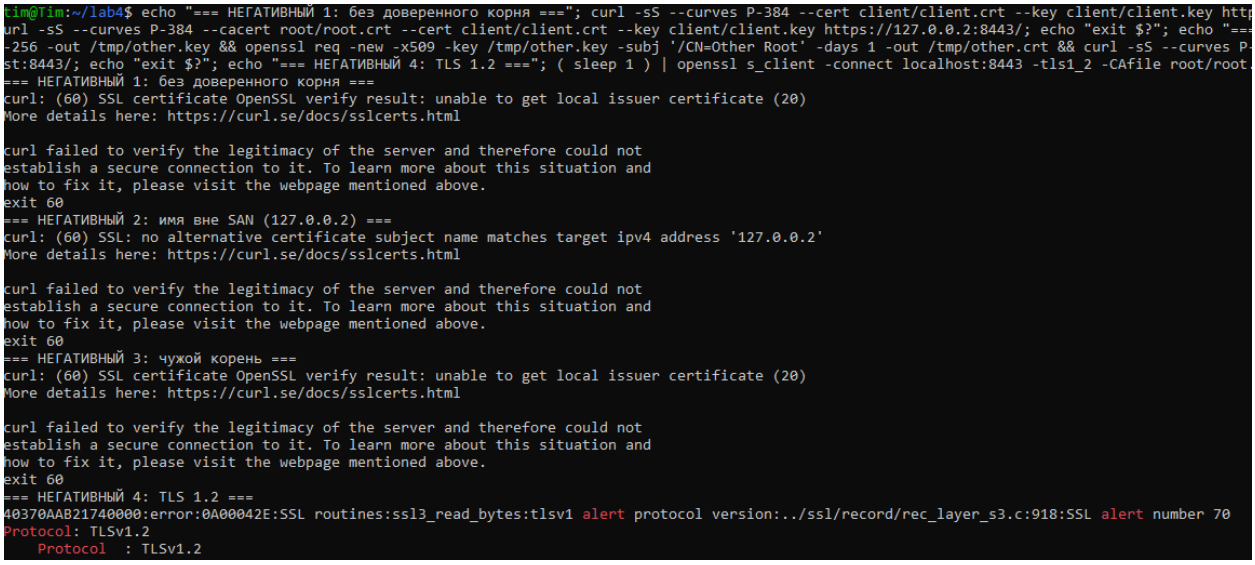
```
tim@Tim:~/lab4$ curl -v --curves P-384 --cacert root/root.crt --cert client/client.crt --ke
tps://localhost:8443/ 2>&1 | grep -E 'SSL connection|TLSv1.3 \(IN\), TLS handshake, (Reques
:|issuer:|verify|< HTTP|tls13 ok'; echo
* TLSv1.3 (IN), TLS handshake, Request CERT (13):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (OUT), TLS handshake, CERT verify (15):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / secp384r1 / id-ecPublicKey
*   subject: CN=localhost
*   issuer: C=BY; O=Lab PKI; CN=Lab Intermediate CA
< HTTP/1.1 200 OK
tls13 ok
```

Соединение установлено по TLSv1.3 с шифром TLS_AES_256_GCM_SHA384 и группой secp384r1, получен ответ HTTP 200 с телом tls13 ok.

**Негативные тесты (во всех передаётся клиентский сертификат, чтобы проверялось именно указанное условие):**

```
# НЕГАТИВНЫЙ: без доверенного корня
```

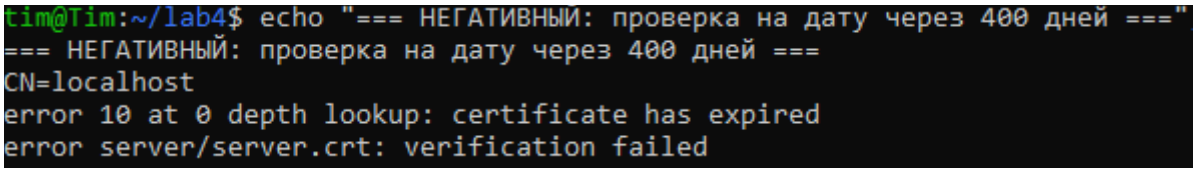
```
curl --curves P-384 --cert client/client.crt --key client/client.key
https://localhost:8443/; echo "exit $?"
# НЕГАТИВНЫЙ: имя вне SAN
curl --curves P-384 --cacert root/root.crt --cert client/client.crt --key
client/client.key https://127.0.0.2:8443/; echo "exit $?"
# НЕГАТИВНЫЙ: чужой корень
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out
/tmp/other.key
openssl req -new -x509 -key /tmp/other.key -subj '/CN=Other Root' -days 1 -out
/tmp/other.crt
curl --curves P-384 --cacert /tmp/other.crt --cert client/client.crt --key
client/client.key https://localhost:8443/; echo "exit $?"
# НЕГАТИВНЫЙ: TLS 1.2
( sleep 1 ) | openssl s_client -connect localhost:8443 -tls1_2 -CAfile
root/root.crt 2>&1 | grep -E 'alert|Protocol'
```




Без корня и с чужим корнем — exit 60, unable to get local issuer certificate. По адресу 127.0.0.2 — exit 60, имя не совпадает с SAN. Попытка TLS 1.2 отвергается сервером: alert protocol version (70).

**Негативный тест: истёкший сертификат (проверка на дату через 400 дней).**

```
openssl verify -attime $(date -d '+400 days' +%s) -CAfile root/root.crt -
untrusted inter/inter.crt server/server.crt
```




Срок действия листа 365 дней, поэтому через 400 дней сертификат считается просроченным.

**Негативный тест: отозванный сертификат (CRL). Отзывается отдельный тестовый сертификат revoked.crt, а не серверный.**

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out
server/revoked.key
openssl req -new -key server/revoked.key -subj '/C=BY/O=Lab PKI/CN=revoked' -out
server/revoked.csr
openssl ca -config inter/ca.cnf -batch -in server/revoked.csr -extfile
server/server.ext -notext -out server/revoked.crt
```

```
openssl ca -config inter/ca.cnf -revoke server/revoked.crt
openssl ca -config inter/ca.cnf -gencrl -out inter/inter.crl
openssl verify -crl_check -CAfile root/root.crt -untrusted inter/inter.crt -
CRLfile inter/inter.crl server/revoked.crt
```

```
tim@Tim:~/lab4$ openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out server/revo
cnf -batch -in server/revoked.csr -extfile server/server.ext -notext -out server/revoked.crt 2
выпуск CRL ==="; openssl ca -config inter/ca.cnf -revoke server/revoked.crt && openssl ca -con
ile root/root.crt -untrusted inter/inter.crt -CRLfile inter/inter.crl server/revoked.crt
Write out database with 1 new entries
Database updated
=== до отзыва ===
server/revoked.crt: OK
=== отзыв и выпуск CRL ===
Using configuration from inter/ca.cnf
Revoking Certificate 1002.
Database updated
Using configuration from inter/ca.cnf
V       271109171459Z           1000    unknown /CN=localhost
V       271109171622Z           1001    unknown /CN=lab-client
R       271109172613Z   261109172613Z   1002    unknown /CN=revoked
=== НЕГАТИВНЫЙ: проверка с CRL ===
CN=revoked
error 23 at 0 depth lookup: certificate revoked
error server/revoked.crt: verification failed
```

После отзыва и выпуска CRL проверка возвращает certificate revoked.

## 8. Захват рукопожатия

Терминал 1 (захват):

```
sudo tcpdump -i lo -w ~/lab4/client/handshake.pcap 'tcp port 8443'
```

```
tim@Tim:~/lab4$ sudo tcpdump -i lo -w ~/lab4/client/handshake.pcap 'tcp port 8443'
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C20 packets captured
40 packets received by filter
0 packets dropped by kernel
tim@Tim:~/lab4$
```

Терминал 2 (клиент с журналом ключей):

```
cd ~/lab4/client
SSLKEYLOGFILE=$HOME/lab4/client/keys.log curl -sS --curves P-384 --cacert
../root/root.crt \
  --cert client.crt --key client.key https://localhost:8443/
sudo chown $USER handshake.pcap
wc -l keys.log
```

```
tim@Tim:~$ cd ~/lab4/client && rm -f keys.log && SSLKEYLOGFILE=$HOME/lab4/client/keys.log curl -sS --curves
rt ../root/root.crt --cert client.crt --key client.key https://localhost:8443/; echo
tls13 ok
tim@Tim:~/lab4/client$ sudo chown $USER handshake.pcap && ls -l handshake.pcap keys.log && wc -l keys.log
[sudo: authenticate] Password:
-rw-r--r-- 1 tim tcpdump 6350 Nov  9 20:29 handshake.pcap
-rw-r--r-- 1 tim tim      938 Nov  9 20:28 keys.log
5 keys.log
tim@Tim:~/lab4/client$
```

## 9. Анализ в tshark

Без журнала ключей:

```
tshark -r handshake.pcap -Y tls -T fields -E header=y -e frame.number -e
tcp.srcport -e tls.record.content_type -e tls.record.opaque_type -e
tls.handshake.type
```

```
tim@Tim:~/lab4/client$ echo "=== БЕЗ КЛЮЧЕЙ ==="; tshark -r handshake.pcap -Y tls -T fields -E header=y -e frame.n
=== БЕЗ КЛЮЧЕЙ ===
frame.number    tcp.srcport     tls.record.content_type tls.record.opaque_type  tls.handshake.type
6       53122   22              1
8       8443    22,20   23,23,23,23,23  2
10      53122   20      23,23,23
11      53122           23
13      8443            23
14      8443            23
16      8443            23
17      53122           23
tim@Tim:~/lab4/client$
```

Без журнала ключей видны только ClientHello (тип 1, кадр 6) и ServerHello (тип 2, кадр 8). Остальные сообщения рукопожатия передаются записями application_data (opaque_type 23): в кадре 8 сервер отправил 5 таких записей, в кадре 10 клиент — 3, но их содержимое и типы не видны. Записи change_cipher_spec (20) отправляются в режиме совместимости и не несут данных. Сопоставление с расшифрованным захватом показывает, что 5 записей сервера — это EncryptedExtensions, CertificateRequest, Certificate, CertificateVerify и Finished, а 3 записи клиента — CompressedCertificate, CertificateVerify и Finished.

С журналом ключей:

```
tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y tls.handshake -T fields -
e frame.number -e tcp.srcport -e tls.handshake.type
tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type == 2'
-T fields -e tls.handshake.ciphersuite -e
tls.handshake.extensions_key_share_group
tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type == 1'
-T fields -e tls.handshake.extensions_server_name
tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type ==
11' -V | grep -E 'dNSName|iPAddress'
tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type ==
15' -T fields -e tcp.srcport -e tls.handshake.sig_hash_alg
cut -d' ' -f1 keys.log | sort -u
```

```
tim@Tim:~/lab4/client$ echo "=== С КЛЮЧАМИ: сообщения ==="; tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y tls.handshake -T fields -E header=y -e f
shark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type == 2' -T fields -e tls.handshake.ciphersuite -e tls.handshake.extensions_key_shar
 -Y 'tls.handshake.type == 1' -T fields -e tls.handshake.extensions_server_name; echo "=== SAN в сертификатах ==="; tshark -r handshake.pcap -o tls.keylog_f
 "=== подпись CertificateVerify ==="; tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type == 15' -T fields -e tcp.srcport -e tls.han
 -u
=== С КЛЮЧАМИ: сообщения ===
frame.number    tcp.srcport     tls.handshake.type
6       53122   1
8       8443    2,8,13,11,15,20
10      53122   25,15,20
13      8443    4
14      8443    4
=== шифр и группа ===
0x1302  24
=== SNI ===
localhost
=== SAN в сертификатах ===
                                GeneralName: dNSName (2)
                                    dNSName: localhost
                                GeneralName: iPAddress (7)
                                    iPAddress: 127.0.0.1
=== подпись CertificateVerify ===
8443    0x0905,0x0906,0x0904,0x0403,0x0503,0x0603,0x0807,0x0808,0x081a,0x081b,0x081c,0x0809,0x080a,0x080b,0x0804,0x0805,0x0806,0x0401,0x0501,0x0601,0x0503
53122   0x0503
=== метки keys.log ===
CLIENT_HANDSHAKE_TRAFFIC_SECRET
CLIENT_TRAFFIC_SECRET_0
EXPORTER_SECRET
SERVER_HANDSHAKE_TRAFFIC_SECRET
SERVER_TRAFFIC_SECRET_0
tim@Tim:~/lab4/client$
```

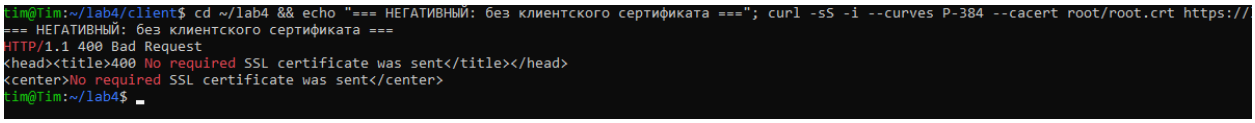
С журналом ключей видны зашифрованные сообщения. Сервер присылает ServerHello (2), EncryptedExtensions (8), CertificateRequest (13), Certificate (11), CertificateVerify (15), Finished (20) и два NewSessionTicket (4). Клиент присылает CompressedCertificate (25), CertificateVerify (15) и Finished (20). Тип 25 (RFC 8879) появился потому, что OpenSSL 3.5 и curl со сборкой brotli/zstd сжимают сертификат; в методичке (OpenSSL 3.0) здесь был бы Certificate (11).

Шифр 0x1302, группа key_share 24 (secp384r1), SNI — localhost, в сертификате сервера dNSName localhost и iPAddress 127.0.0.1. Для сервера в колонке sig_hash_alg сначала выводится список signature_algorithms из CertificateRequest, а последнее значение 0x0503 (ecdsa_secp384r1_sha384) — подпись CertificateVerify; клиент также подписал 0x0503. Журнал ключей содержит метки CLIENT_HANDSHAKE_TRAFFIC_SECRET, SERVER_HANDSHAKE_TRAFFIC_SECRET, CLIENT_TRAFFIC_SECRET_0, SERVER_TRAFFIC_SECRET_0, EXPORTER_SECRET.

## 10. Задача варианта: mTLS

### Негативный тест: запрос без клиентского сертификата.

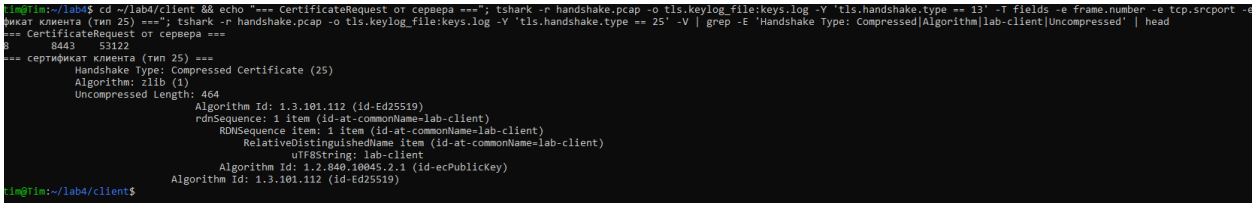
```
curl -sS -i --curves P-384 --cacert root/root.crt https://localhost:8443/ | grep
-E 'HTTP/|No required'
```




Без клиентского сертификата nginx отвечает HTTP 400 No required SSL certificate was sent. TLS-рукопожатие при этом завершается, а отказ формируется уже на уровне HTTP, поэтому клиент получает страницу ошибки, а не TLS-alert.

Сообщения mTLS в захвате:

```
tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type ==
13' -T fields -e frame.number -e tcp.srcport -e tcp.dstport
tshark -r handshake.pcap -o tls.keylog_file:keys.log -Y 'tls.handshake.type ==
25' -V | grep -E 'Handshake Type: Compressed|Algorithm|lab-client|Uncompressed' |
head
```




Сервер (порт 8443) в кадре 8 после EncryptedExtensions присылает CertificateRequest (тип 13), в котором перечисляет допустимые алгоритмы подписи. Клиент в кадре 10 отвечает своим сертификатом в сообщении CompressedCertificate (тип 25, RFC 8879): сертификат сжат алгоритмом zlib, размер после распаковки 464 байта. Wireshark распаковал сообщение: внутри сертификат CN=lab-client с открытым ключом ECDSA (ecPublicKey), подписанный промежуточным CA алгоритмом Ed25519. Затем клиент отправляет CertificateVerify (тип 15) с подписью 0x0503 (ecdsa_secp384r1_sha384) — этим он доказывает владение закрытым ключом сертификата. Сервер проверяет цепочку клиента по файлу ca-chain.pem (промежуточный CA и корень) и только после этого отвечает HTTP 200. Все эти сообщения передаются зашифрованными и видны только с журналом

ключей. Без клиентского сертификата TLS-рукопожатие завершается, но nginx отклоняет запрос уже на уровне HTTP ответом 400 No required SSL certificate was sent, а не TLS-alert.

## Вывод

В ходе лабораторной работы на собственном тестовом CA была выпущена цепочка сертификатов X.509 с расширениями (корневой и промежуточный CA на Ed25519, серверный и клиентский сертификаты на ECDSA P-384), настроен HTTPS-сервер nginx, принимающий только TLS 1.3 и требующий сертификат клиента (mTLS), проверены цепочка доверия и отзыв сертификата через CRL, захвачено рукопожатие TLS 1.3 и разобрано в tshark, в том числе после расшифровки через журнал ключей SSLKEYLOGFILE.